COMS W4111 · PROJECT 1, PART 3

Flask + PostgreSQL

TODAY

Topics

1

Starter code & routing

Run the server, map URLs to functions

2

SQL with SQLAlchemy

create_engine, text(), reading rows. No ORM.

3

Parameterized queries

Keep user input out of your SQL text

4

Templates & forms

Show results in Jinja, accept input

PART 1 · SETUP

Running the starter code locally

What's in the repo

File	Role
server.py	Routes + SQL
templates/	Jinja HTML
static/	CSS
requirements.txt	Packages
.env	Your secrets

Don't commit .env or proxy-key.json

```
# 1. environment
python3.14 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# 2. .env (repo root)
DB_USER=your_uni
DB_PASSWORD=your_password
# 3. proxy, in a second terminal
./cloud-sql-proxy --port=5454 \
  --credentials-file=~/proxy-key.json \
  <instance-connection-name>
# 4. run → http://127.0.0.1:8111
python server.py
```

PART 1 · ROUTING

A route maps a URL to a function

```
app = Flask(__name__)
@app.route("/")
def index():
    return "Hello, W4111!"

@app.route("/hello/<name>")
def hello(name):
    return render_template_string(
        "Hello, {{ name }}!", name=name)

@app.route("/square/<int:n>")
def square(n):
    return f"{n} squared is {n * n}"
```

Try it

URL	Result
/	Hello, W4111!
/hello/ada	Hello, ada!
/square/7	7 squared is 49
/square/abc	404

PART 1 · REQUESTS

Reading input and returning pages

The route

```python
# /search?q=sql
q = request.args.get("q", "")

# GET shows the form, POST receives it
@app.route("/echo", methods=["GET", "POST"])
def echo():
    if request.method == "POST":
        msg = request.form["message"]
        return redirect(url_for("hello", name=msg))
    return render_template("step1_form.html")
```

templates/step1_form.html

```html
<form method="POST">
  <input name="message">
  <button>Send</button>
</form>
```

request.args: query string (GET)
request.form: fields sent with POST
url_for: builds a URL from a function name

PART 2 · CONNECTING

One engine, one connection per request

```
url = URL.create("postgresql+psycopg2",
    username=os.environ["DB_USER"],
    password=os.environ["DB_PASSWORD"],
    host="127.0.0.1", port=5454, database="proj1part2")
engine = create_engine(url)

@app.before_request
def before_request():
    g.conn = engine.connect()

@app.teardown_request
def teardown_request(exc):
    g.conn.close()
```

Engine

A pool of connections. Create it once.

Connection

Borrowed for one request, stored on g

close()

Returns it to the pool and rolls back anything uncommitted

PART 2 · RUNNING SQL

conn.execute(text("..."))

```
# one value
count = conn.execute(text("SELECT count(*) FROM students")).scalar_one()

# many rows: loop over the result
cursor = conn.execute(text("SELECT uni, name, major FROM students"))
for row in cursor:
    print(row[0], row.name, row.major)  # by index or by name
cursor.close()

# writes (INSERT / UPDATE / DELETE) need a commit
conn.execute(text("INSERT INTO students VALUES ('gh5678', 'Grace Hopper', 'CS')"))
conn.commit()
```

PART 2 · CURSORS

A result can be read only once

```
cursor = conn.execute(text("SELECT name FROM students"))
first  = [row.name for row in cursor]
second = [row.name for row in cursor]

# first  → ['Ada Byron', 'Grace Hopper', 'Alan Turing']
# second → []   already used up
```

Need the rows twice, or in a template? **Turn them into a list first.**

Ways to read a result

Call	Gives you
.scalar_one()	One value
.fetchone()	One row, or None
.fetchall()	List of rows
.scalars().all()	List of the first column
.mappings().all()	List of dict-like rows

PART 2 · THE RULE

No ORM: you write every SQL query yourself

Allowed

```
create_engine(...)
engine.connect()
conn.execute(text("SELECT ..."))
conn.commit()
```

Not allowed

```
class Student(Base): ...
session.query(Student)
select(students).where(...)
Flask-SQLAlchemy models
```

SQL your app runs should be visible in your code.

PART 3 · DON’T DO THIS

Pasting input into SQL lets users rewrite your query

```
user_input = request.form["username"]   # nobody' OR '1'='1

sql = f"SELECT * FROM accounts WHERE username = '{user_input}'"
conn.execute(text(sql))
```

What the database actually runs:

```
SELECT * FROM accounts WHERE username = 'nobody' OR '1'='1'
```

The condition is always true, so it returns **every account**. The same trick can run UPDATE or DROP TABLE.

PART 3 · DON'T DO THIS

It can also run a second statement

```
evil_input = "x'; DROP TABLE victim; --"

sql = f"SELECT * FROM accounts WHERE username = '{evil_input}'"
conn.execute(text(sql))
```

What the database actually runs:

```
SELECT * FROM accounts WHERE username = 'x'; DROP TABLE victim; --'
```

' closes the string, **;** ends the SELECT, **--** comments out the leftover quote. The table is gone.

PART 3 · DO THIS

Send the SQL and the values separately

```
conn.execute(
    text("SELECT * FROM accounts WHERE username = :username"),
    {"username": user_input},
)   # → []  no user is literally named "nobody' OR '1'='1"

# INSERT works the same way, from server.py:
conn.execute(text("INSERT INTO test(name) VALUES (:name)"), {"name": name})
conn.commit()
```

:name in the SQL

A placeholder, never inside quotes

A dict of values

Second argument to execute

The driver handles it

Quotes like O’Brien just work

PART 3 · SIDE BY SIDE

Same input, two ways

user_input = nobody' OR '1'='1

f-string

1. You write

```
f"... WHERE username = '{user_input}'"
```

2. Postgres receives

```
... WHERE username = 'nobody' OR '1'='1'
```

3. Result

Every row. The input became SQL.

Parameter

1. You write

```
text("... WHERE username = :username"),
{"username": user_input}
```

2. Postgres receives

```
... WHERE username = 'nobody'' OR ''1''=''1'
```

3. Result

No rows. The input stayed one string.

PART 3 · TWO COMMON CASES

LIKE searches and column names

Wildcards go in the value

```
search = request.form["search"]  # b
pattern = f"%{search}%"  # "b" → "%b%"
conn.execute(
  text("SELECT name FROM students"
       " WHERE name ILIKE :pattern"),
  {"pattern": pattern},
)
```

The f-string builds the **value**, not the SQL, so it's quoted like any parameter.

Names can't be parameters

A parameter always arrives as a quoted string:

```
ORDER BY :col   → ORDER BY 'name'   error
FROM :table     → FROM 'students'   error
```

Instead, check the name against a fixed list:

```
sort = request.args.get("sort")
if sort not in {"name", "major"}:
    sort = "name"
conn.execute(text(
f"SELECT * FROM students ORDER BY {sort}"))
```

PART 4 · TEMPLATES

From query result to HTML table

step4_app.py

```
@bp.route("/")
def students():
    cursor = g.conn.execute(text(
        "SELECT uni, name, major"
        " FROM tutorial_students"))
    rows = cursor.mappings().all()
    cursor.close()
    return render_template(
        "step4_students.html",
        rows=rows)
```

templates/step4_students.html

```
<table>
  <tr><th>UNI</th><th>Name</th></tr>
  {% for row in rows %}
  <tr>
    <td>{{ row.uni }}</td>
    <td>{{ row.name }}</td>
  </tr>
  {% endfor %}
</table>
```

Pass a list, not the cursor. **{{ ... }}** escapes HTML automatically, so a name like <script> shows as text.

PART 4 · FORMS

GET to search, POST to change data

Search: method="GET"

```
<form method="GET">
  <input name="q">
</form>
# in the route
q = request.args.get("q", "")
g.conn.execute(text(
  "... WHERE name ILIKE :pattern"),
  {"pattern": f"%{q}%"})
```

Add: method="POST"

```
@bp.route("/add", methods=["POST"])
def add_student():
    name = request.form.get("name", "")
    if not name: abort(400)
    g.conn.execute(text("INSERT ..."),
        {"name": name, ...})
    g.conn.commit()
    return redirect(url_for("step4.students"))
```

- The input's **name** is the key in request.form.
- **Validate**, then INSERT and **commit**.
- **Redirect** after POST, so a refresh doesn't add the row twice.